

Procedural Sword Generation

Alex O'Nions

BSc (Hons) Computer Games Applications
Development, 2026

School of Design and Informatics
Abertay University

Table of Contents

Table of Figures	iii
Table of Tables	iv
Acknowledgements	v
Abstract.....	vi
Abbreviations, Symbols and Notation	vii
Chapter 1 Introduction	1
1.1 The Need for Procedural Generation in Games	1
1.2 The Use of Procedural Generation in Games.....	1
1.3 Aim	2
1.4 Research Question	2
1.5 Research Question Critical Analysis.....	2
1.6 Objectives	3
Chapter 2 Literature Review	3
2.1. Procedural Generation Techniques	4
2.1.1. Modular Components Generation	4
2.1.2. Noise Driven Generation.....	5
2.1.3. Parametric/Rule Based Generation	7
2.1.4. Spline Based Silhouette Generation	7
2.2. Swords	8
2.3. Swords In games.....	11
2.4. Procedural Sword Generation	12
Chapter 3 Methodology.....	13
3.1. Overview	13
3.2. Values	14
Chapter 4 Results	26
4.1 Visual Questionnaire Data.....	27
4.2 Program Questionnaire Data.....	30

Chapter 5 Discussion	33
5.1. Survey Session	33
5.2. Visual Questionnaire Data Analysis	33
5.3. Program Questionnaire Data Analysis	35
5.4. Project Findings.....	37
5.5. Summary	38
5.6. Evaluation.....	38
Chapter 6 Conclusion and Future Work.....	39
6.1. Main Conclusion.....	39
6.2. Implications	40
6.3. Future Work.....	41
References.....	43

Table of Figures

Figure 1: Creating A Sword Silhouette

Figure 2: Extruding The 2D Profile

Figure 3: Sharpening The Edge

Figure 4: Bevel Variations

Figure 5: Converging The Tip of The Blade

Figure 6: Generated Sword Images

Figure 7: Visual Rating of Sword Results

Figure 8: Pie Charts of Generated Swords Believability

Figure 9: Blade Creation Pie Results

Figure 10: Randomisation Results Pie Chart

Table of Tables

Table 1: Questionnaire Survey Table

Table 2: Program Survey Table

Acknowledgements

I would like to thank my supervisor Gareth Robinson for all the guidance they have provided me throughout the year. Additionally, I would like to thank all the staff at Abertay who have provided support and put together the module.

Abstract

Context: Procedural asset generation is becoming increasingly important in game development, enabling the creation of customizable and unique content at scale. However, generating complex meshes, such as swords, can present unique challenges, particularly at the blade tip, where multiple vertices must converge into one point, and along the blade's curvature, which must appear smooth while maintaining visual realism. This project addresses the need for visually polished, customizable sword meshes suitable for game development.

Aim: The Aim of this project is to use procedural mesh generation system to produce customizable sword models with logical tip convergence, smooth curvature, and realistic visual appearance for game development.

Method: A prototype has been developed using procedural generation techniques. The system allows users to change parameters which will affect the final output of the blade. This was then evaluated by having users visually compare generated and 3D modelled swords, as well as letting users test the program and give feedback.

Results: The results collected reveal the effectiveness of the implemented system and show how affective procedural generation can be for creating assets in games. The areas of focus are visual appeal, realism and user satisfaction. The results of this research show that procedural generation is viable for content generation, though this version does need some refinement if it was to be used on a larger scale.

Conclusion: This project helps conclude whether using procedural generation for creating game assets is viable, compared to usual methods like Hand 3D Modelling, considering visual results as well as the time it saves when making a model.

Abbreviations, Symbols and Notation

Procedural Generation (PG)

Procedural Content Generation (PCG)

User Interface (UI)

Chapter 1 Introduction

1.1 The Need for Procedural Generation in Games

Creating content for games is a costly process as it requires highly trained skillsets to do so and over a long period of time. Cyberpunk 2077 (CD Projekt Red, 2020) had a base game with a development cost of over \$130 million (McEvoy, 2024), and Genshin Impact (miHoYo, 2020) has cost over \$900 million in production costs (Wood, 2024). The use of procedural content generation in both scenarios could reduce the time of development thus reducing the overall cost, this can be done by giving the artists a generated base template or purely generating assets themselves. The downside to this is that creating the initial PG Software is a tricky process and needs a lot of fine tuning, especially for the use in games (Shaker, Togelius & Nelosn, 2016).

1.2 The Use of Procedural Generation in Games

Procedural Generation is widely used in modern game development typically being used in open world games to create large, dynamic and replayable worlds for players (Hendrikx et al., 2013). Whilst it is mostly associated with open world generation its use goes far beyond environments. Many games, such as Borderlands (Gearbox Software, 2009), Minecraft (Mojang Studios, 2011) and No Mans Sky (Hello Games, 2016), use procedural systems to generate enemies, loot, weapons, quests and even entire stories, which allows developers to produce content in less time, whilst allowing for more variety in their games.

A well-known example of this is Borderlands (Gearbox Software, 2009) which introduced a procedural weapon system. The most recent entry Borderlands 4 (Gearbox Software, 2025) demonstrates the scale of this approach, by producing billions of gun variations (McCrae, 2025). This is done with premade components which include barrels, stocks, magazines, scopes and elemental modules into thousands of possible

combinations. Although this system creates an impressive variety, each weapon still ultimately uses the same limited pool of hand-crafted parts, meaning many guns share similar silhouettes and design.

Instead of assembling weapons from pre hand crafted pieces, it is possible to procedurally generate the 3D models themselves, allowing for each weapon to have its own entirely unique shape, proportions and silhouette.

This approach would enable each weapon to feel truly distinct, rather than just being a modified template, as well as being unlike any other the player has seen before. Additionally, once a weapons geometry has been defined, additional traits, such as weight, attack patterns, and effects can be dynamically defined based on its shape and size. This would result in the weapon not just being visually unique but to also have unique in-game properties.

1.3 Aim

This project aims to develop a prototype system which can procedurally generate at runtime, which can be used as 3D assets in games or other 3D rendered applications.

1.4 Research Question

Can procedural content generation be used for the algorithmic creation of swords, while maintaining geometric clarity and clear function?

1.5 Research Question Critical Analysis

Procedurally generating the 3D Mesh itself for weapons in games is a feature which has been looked at in mainstream games. The most important part is the system producing consistent and viable swords, as this will allow players to feel as if every weapon they encounter is unique.

Thus, allowing for gameplay to feel less repetitive and potentially more exciting, as players hunt for their perfect sword.

The proposed question intends to identify whether procedural content generation can be used specifically for the weapons players might use and if it can be as viable or exceed traditional 3D modelling methods, allowing for time to be saved on games overall production.

1.6 Objectives

The primary objectives of the project are:

- Research sword designs and functionality
- Research the historical importance of different swords
- Design a procedural framework for sword generation using splines, vertex manipulation, interpolation logic and parameter-driven logic
- Design a procedural framework for sword textures/materials, using Unity's Shader Graph.
- To allow the user to customise the parameters to create unique weapon designs.
- To evaluate the project's effectiveness, the quality and visual realism of the generated blades through user feedback, testing and comparison to historical swords.
- To reveal the scalability and efficiency potentials of procedural mesh generation compared to traditional mesh creation methods.

Chapter 2 Literature Review

To support the successful development of this project, research was undertaken into procedural generation techniques as well as the history of creating and designing a sword. This literature review examines key sources that inform the project methodology, showing how the project fits within existing research and identifying the foundations upon which the project is built.

Procedural Generation is the creation content through algorithms rather than manual design, often using a pre-defined process and some randomness defined by the programmer, this usually occurs at runtime though can also occur during development (Yannakakis & Togelius,

2011). Procedural Content Generation has been used in games since the late 1970's and early 1980's first appearing in titles such as *Rogue* (Toy, Winchman & Arnold, 1980), which generated dungeon layouts algorithmically and later *Elite* (Braben & Bell, 1984), which used procedural methods to create entire galaxies within strict memory constraints. Over time, its use has expanded into terrain, items, quests, characters and even entire worlds (Shaker, Togelius & Nelson, 2016). Examples of this expansion can be seen in modern games such as *Borderlands* (Gearbox Software, 2009) which uses procedural systems to generate millions of unique weapons, *Minecraft* (Mojang Studios, 2011) which procedurally generates terrain and biomes, and *No Mans Sky* (Hello Games, 2016) which creates entire planets, ecosystems, and galaxies algorithmically.

This method of generating content algorithmically, rather than sorting it manually, was especially effective as early computers had a very limited memory, and generating content algorithmically required far less storage than saving many hand-crafted assets (Braben & Bell, 1984).

This also allowed games to present an effectively endless stream of new content, ensuring that players could continue to engage with experiences that remained fresh across repeated playthroughs (Hendrikx et al., 2013).

2.1. Procedural Generation Techniques

2.1.1. Modular Components Generation

Modular Component Generation is the process of constructing complex objects through the assembly of prefabricated parts. Rather than creating each finished asset from scratch. This approach enables rapid variation because new assets can be produced simply by recombining existing components, rather than modelling each item from scratch. At the same time, consistency is maintained because all the components are designed

within the same stylistic framework, ensuring that any combination still aligns with the intended visual identity (Smelik et al., 2014). This is widely used in game development for environments, weapons and character equipment (Shaker, Togelius & Nelosn, 2016), and remains a common approach within procedural modelling research due to its flexibility when generation 3D content (Smelik et al., 2014).

In the context of sword creation, swords can be created by combining, blades guards, grips and pommels from a pre-defined library of artist designed components. By reducing the amount of variation to a variety set of parts, the system can generate numerous sword configurations that all reinforce the same visual identity. Artists also retain full control over the form and style of each component, ensuring that even procedurally assembled results remain coherent and aligned with the project's aesthetic goals. While this method does not aim for complete uniqueness in every output, its structured variation supports stylistic consistency and efficient asset production, making it well-suited to projects that prioritise a unified artistic direction.

2.1.2. Noise Driven Generation

Noise Driven Generation refers to the use of mathematical noise functions – these include but are not limited to Perlin (Perlin, 1985), Cellular (Worley, 1996), and Simplex (Perlin, 2002) Noise to produce variations in shapes, textures and geometry (Smelik et al., 2014).

Noise provides a way of generating randomness that remains structurally coherent, assigning values across space in a way that avoids the harsh lack of consistency that pure randomness can create. Different noise types achieve this in different ways, gradient based methods like Perlin and Simplex Noise, interpolate between random gradients to create smooth natural patterns. Whilst distance-based methods like Cellular

noise, create values from proximity feature points, producing cell like structures. Together these functions provide controllable variation suitable for shaping forms and adding natural details.

These functions allow for smooth generation with the use of pseudo random patterns that can be manipulated to control spatial distribution silhouette variation and material properties.

Noise-based methods are used in PCG because they provide coherent, structured variation, without the need of premade assets, making them suitable for producing, terrain textures, items and organic forms (Hendrikx et al., 2013). Noise forms the foundation of many procedural modelling techniques (Smelik et al., 2014), due to its ability to create smooth natural looking patterns.

In the context of sword creation, noise functions can be used to create the blade's silhouette and profile by generating smooth, natural variations along its outline, allowing the overall shape to change in width, curvature and taper (Ebert et al., 2003).

Perlin Noise can be particularly useful for generating smooth curvature along the cutting edge of the blade, allowing for variety in width and flow whilst maintaining a stylistic appearance (Perlin, 1985). Simplex Noise can be used to refine these forms with a more stable pattern making it useful for consistent output over a large set of swords (Perlin, 2002). Whilst Cellular Noise offers additional surface possibilities such as distinct material irregularities or patterns within the shader (Worley, 1996). While sharper feather like chips and jagged edges, require additional control beyond what noise naturally provides, this isn't an issue for projects aiming to consistently create polished and uniform designs, compared to projects aiming for more worn and used swords. As a result, noise remains a reliable option for introducing curvature, width variation and material details within the sword creation process.

2.1.3. Parametric/Rule Based Generation

Parametric and rule-based generation refers to the process of creating assets based on a set of parameters or rules that determine the structure, proportions and behaviour of the final output (Shaker, Togelius & Nelosn, 2016). In this approach the system or user specifies values such as length, curvature, width, symmetry and other stylistic constraints, and the generator produces its results based on these pre-defined inputs. Unlike modular systems, which rely on assembling premade components, parametric methods construct the asset procedurally by manipulating algorithmic relations between features (Smelik et al., 2014).

Rule based systems extend this idea by introducing conditional logic, allowing the generator to enforce design constraints such as blade width, silhouette coherence and historically consistent proportions. This makes parametric and rule-based approaches effective for assets that require structural consistency and controlled variation, as the rules ensure that generated sword remain functional, and stylistically (Smelik et al., 2014).

This project uses parametric and rule-based techniques as the core part of its generative process. They provide the level of precision needed to control the shaping of the sword's silhouette, ensuring that variations remain within acceptable design boundaries, whilst still allowing for the necessary diversity across the generated models.

2.1.4. Spline Based Silhouette Generation

Spline based silhouette generation involves using mathematical splines, such as Bezier curves and Catmull-Rom splines to define smooth, continuous curves through a series of control points (Bartels, Beatty & Barsky, 1983). In many procedural workflows, splines are used to create

the silhouette of an object, or they act as a structural guide that help inform the final generated shape (Smelik et al., 2014).

Spline based approaches are particularly effective for assets where the outline is driven by an underlying path (Smelik et al., 2014). A common strategy is to define a central axis using a spline, which acts as the foundation from which secondary properties such as width, taper and silhouette are derived (Bartels, Beatty & Barsky, 1983). This enables consistent control over the curvature whilst maintaining smoothness across the generated geometry.

In Summary Spline based methods provide a robust foundation for procedural silhouette construction, as the underlying curve defines the overall flow and curvature of the form. By manipulating the splines control points, a wide variety of coherent shapes can be produced whilst maintaining smoothness and structural consistency. This makes spline driven workflows particularly well suited to systems that required controlled variation and clear, editable 2D outlines that can be used as the basis for 3D generation

2.2. Swords

Swords come in lots of shapes and sizes, with each having their own history, culture and distinctive features (Oakeshott, 1964).

When looking at swords we first need to look at the individual parts that make up a sword, these include the Hilt, Guard, Grip, Pommel, Fullers, Spine, Bevels, Edges and Tip.

The Hilt consists of three main parts, the Guard, Grip and Pommel, each of which serves a distinct structural and functional purpose (Oakeshott, 1964). The guard provides protection for the wielder's hands, by preventing other blades from sliding down to their fingers (Clements, 1998). The Grip provides the primary point of contact between the wielder

and the weapon, usually being constructed of wood, leather or metal, all of which affect the shock absorption and friction felt when the sword is in use (Clements, 1998). The pommel, meanwhile, acts as a counterweight to the blade, helping balance the weapon and improve manoeuvrability (Clements, 1998). In many designs it also serves as an anchor for the blade and in some cases, as an additional striking surface. Ultimately, the hilt integrates these components into a cohesive structure that protects the wielder, enhances control and ensures effective manoeuvrability during use (Oakeshott, 1991).

Fullers consist of one or more vertical grooved forged or ground into the blade, each of which serve their own structural and functional purposes (Williams, 2012). Their primary roles are to reduce the overall weight of the blade, without compromising its strength, allowing the weapon to become easier to wield (Williams, 2012). By redistributing material away from the centre and towards the edges, fullers increase the blades overall stiffness, relative to its mass, improving the wielders handling and reducing possible fatigue overtime (Williams, 2012). In addition to these physical bonuses, fullers can also contribute to the sword's aesthetics, with the number of fullers, their depth and placement varying across cultures and historical periods (Oakeshott, 1991). Ultimately, fullers enhance both the structural efficiency and visual character of the blade, reducing weight, whilst preserving rigidity and contributing to the blades handling and aesthetic identity (Williams, 2012; Oakeshott, 1991).

The spine is the core structural ridge, running along the centre of double-edged blades and the back of single edge blades. Providing the primary source of its strength and rigidity (Oakeshott, 1991). As the thickest part of the blades cross section, it resists bending and distributes the stressed generated during use (Oakeshott, 1991). Variations in the thickness and profile, such as flat, rounded or reinforced spines, directly influence the weapon's durability, flexibility, and overall handling characteristics (Oakeshott, 1991). Ultimately, the spine acts as the blade's backbone,

forming the central structural ridge, that ensures the edges can perform effectively without deforming under stresses of impact (Oakeshott, 1991).

Bevels form the angled surfaces that run along each side of the blade, converting to create the cutting edge (Clements, 1998). Their geometry plays a crucial role in determining how the weapon interacts with the target, influencing durability, sharpness and the cutting characteristics of the blade (Williams, 2012). Narrow bevels produce finer sharper edges suited for slicing and precise cutting, however, are more vulnerable to chipping (Williams, 2012). In contrast, broader bevels distribute force across a larger surface area, increasing edge resilience and making the blade better suited for contact with harder materials (Clements, 1998). The depth, angle and consistency of the bevel all affect the blades weight distribution and handling as well as the ease in which the blade can be maintained over time (Oakeshott, 1991). Ultimately, bevels define the balance between sharpness and strength, shaping how effectively a blade cuts, withstands impact and performs during use (Williams, 2012; Clements, 1998).

Edges form the primary cutting surfaces of the blade and are created by tapering the material from the spine towards a narrow-sharpened line (Oakeshott, 1991). The edge geometry determines how effective the weapon is at slicing, cleaving or piercing, based on factors such as the bevel angle, and edge thickness, both of which influence the blades sharpness and durability (Williams, 2012). Narrower more acute edges provide finer cutting performance but are more likely to deform or chip (Williams, 2012). In contrast thicker edges offer more resilience making them better suited for harder targets or repeated impact (Clements, 1998). The edges, play a crucial role in the blades handling, as their shape contributes to the weight distribution and the transition between the spine and the cutting surface (Oakeshott, 1991). Ultimately, the edges represent the functional interface between the weapon and its target, balancing sharpness, strength and longevity based on the blade's intended purpose (Williams, 2012; Clements, 1998).

The tip forms the blade's point and is responsible for concentrating force into a small, focused area, enabling for effective thrusting and penetration (Clements, 1998). Its geometry varies significantly across different blade styles, with needles like points optimised for piercing gaps in armour, to broader or reinforced tips which are designed to withstand repeated impacts without bending or fracturing (Oakeshott, 1991). The angle, thickness and alignment of the tip relative to the blade's central axis, all have influence over the structural integrity and efficiency in which momentum is transferred during a thrust (Williams, 2012). As the blade tapers towards its point, the tip must balance sharpness and durability, ensuring it can perform precise thrusting actions without compromising the sword's durability (Williams, 2012). Ultimately, the tip extends the blade's functional role, enabling controlled thrusts alongside cutting actions, contributing to the weapon's effectiveness across combat scenarios (Clements, 1998; Oakeshott, 1991).

2.3. Swords In games

Swords have played a central role in games for decades, appearing across many genres. These games range from medieval games like *Kingdom Come: Deliverance* (Warhorse Studios, 2018), *For Honor* (Ubisoft, 2017), and *Chivalry 2* (Torn Banner Studios, 2021), that rely on weapons which closely reflect real world forged and culturally unique swords, as well as more futuristic games including *Halo* (Bungie, 2001), *Warframe* (Digital Extremes, 2013), *Destiny* (Bungie, 2017), which often consist of more imaginative sword designs that aren't historically accurate. As a result, modern day game development requires a constantly evolving variety of sword models, spanning from historical reconstructions to more exaggerated fantasy weapons, and fully imaginative futuristic blades.

Traditionally, these assets are created by 3D artists who hand model each component of the sword (Ahearn, 2013). Whilst this approach allows for full artistic control, it becomes more impractical as the variety of swords needed grows or the if there are a high volume of swords. Manual creation also takes a considerable amount of time and iterations. These challenges highlight the potential advantages procedural content generation (PCG) offers (Shaker, Togelius & Nelosn, 2016). By automating aspects of the modelling process PCG can support rapid iteration and creation of a wide variety of swords, reducing the need for manual workload, enabling developers to meet the ever-growing demand for a wide variety of unique swords in modern games (Yannakakis & Togelius, 2018)

2.4. Procedural Sword Generation

Procedural Sword Generation has been explored in interactive systems such as Volcano: Interactive Sword Generator (Daniele Loiacono, 2015). Volcano allows users to manipulate a set of high-level parameters such as blade length, curvature, guard shape and overall silhouette, to generate a wide variety of swords. At a technical level, Volcano operates by adjusting predefined geometric templates according to user selected sliders, blending between stored shape profiles to produce new silhouettes. This makes it highly effective for rapid visual exploration, enabling users to experiment with multiple variations without requiring manual modelling.

Volcano's strengths lie in its ability to generate distinct silhouettes whilst maintaining stylistic coherence. It provides a useful point of comparison for procedural weapon systems, demonstrating how user guided parameters can be combined with algorithmic variation to produce a visually diverse range of results. It also offers valuable visual reference material for assessing whether a sword appear historically or visually plausible.

However, Volcano's generative scope is limited to high level shape variation. The system does not attempt to model the detailed geometric structure of the sword, such as bevels, edge profiles, fuller placement, cross sectional geometry or tip construction. Its outputs remain as primarily 2D silhouettes or simplified 3D forms, rather than accurate game ready models. As a result, whilst Volcano excels at stylistic experimentation, it does not address the finer structural features required for functional or realistic sword assets.

This project extends the existing literature by focusing on these missing geometric details. Unlike Volcano, which modifies broad shape templates, this work introduces a parametric rules-based approach capable of generating refined blade geometry, including bevel angles, edge thickness, curvature profiles and tip structures. By modelling the underlying anatomy of the sword rather than only its outline, this project contributes a more detailed and technically grounded method for procedural sword generation.

Chapter 3 Methodology

3.1. Overview

As discussed in the literature review, existing procedural weapon systems such as Volcano, primarily focus on high level shape variation. This project therefore evaluates whether more granular geometric parameters can produce swords that match the visual quality of hand modelled assets.

To investigate the research question, the project requires a prototype application capable of generating visually appealing swords suitable for use in games.

The application primary purpose is to generate swords that are both historically accurate and visually appealing whilst being aesthetically

comparable to Hand-modelled 3D swords, allowing for procedural generation to then be evaluated as a viable option for reducing development time, without compromising quality.

The system is created using C# within Unity, which provides an accessible environment to visualise the generated swords, and adjust parameters through Unity's built-in UI.

To assess if the system is viable, the generated blades were compared against Hand Modelled 3D swords by participants, who evaluated their visual quality and realism. Additional user testing focused on ease-of-use testing and to see if the system's ability to match the users desired sword. These evaluations collectively inform whether the procedural approach can server as a practical alternative to traditional modelling workflows.

3.2. Values

This procedural system exposes a set of parameters that control the blade's geometry, structure and visual characteristics of each generated sword. These parameters were selected as they correspond directly to anatomical features identified in real sword designs, as well as within literature that defines a sword's structural and functional behaviour (Oakeshott, 1991; Williams, 2012). By exposing these values, the system ensures that generated swords remain grounded in both historical and mechanically functional geometry.

Geometric Parameters:

- **Blade Height/Length** – Controls the overall size of the sword, ensuring generated blades fall within plausible proportions.
- **Width Min/Max** – Defines how the blade widens or narrows along its width, used to create straight tapered or flared silhouettes.
- **Thickness** – Sets the depth of the blade, used to maintain structural plausibility, and scale bevel depth appropriately.

- **Spline Count** – Determines the number of control points used in the spline, influencing how detailed the silhouette can be and how smoothly curvature is represented.
- **Segment Angle** – Controls the amount of angular variation applied to each blade segment, introducing subtle random deformation along the edges.
- **Tip Mode** – Controls whether the blade's tip converges symmetrically along the centre axis or is offset to one side, affecting the style of the point.
- **Quality** – Controls the resolution of the blade generation process, influencing the smoothness of curves, the accuracy of geometric features and the overall fidelity of the final mesh.

Curvature Parameters:

- **Max Curvature** – Controls the maximum bend along the blade's spine, determining how straight or curved the overall silhouette is.
- **Peak Factor** – Scales the intensity of the applied curvature, allowing the blade's arc to be exaggerated or reduced.
- **Step Size** – Controls the interpolation along the curvature of the blade, with smaller step sizes producing cleaner transitions and smoother edges.
- **Straight Segment Threshold** – Specifies the length of the initial straight section at the base of the blade, before curvature begins to be applied along the spline.

Edge Parameters:

- **Edge Sharpness** – Controls how far out the blade's edges are extended across the width to create a sharper edge profile.
- **Nick Amount** – Determines how many nicks are applied to the blade's edges.
- **Edge Side** – Specifies whether the blade is single edged or double edged by defining which side/sides have a sharpened edge.

- **Collapse Mode** – Determines which side of the blade collapses relative to the centreline, controlling the direction of the bevel's deformation.
- **Spine Offset** – Determines the horizontal displacement of the spine relative to the blade's centreline.
- **Bevel thickness** - Determines the thickness of the blade immediately before the cutting edge.

Cross Section Parameters:

- **Start Height** – Determines the position along the blade's length, where the cross section begins to take effect, defining the starting point of the profile transition.
- **End Height** - Determines the position along the blade's length, where the cross-section influence ends, marking the final point of the profiles transition.
- **Profile Scale** – Controls the overall scale or thickness of the cross section, influencing the blades structural and visual proportions.
- **Transition Curve** – Defines how smoothly the cross-section blend between the start and end points, shaping the rate of the profiles transformation.

Fuller Parameters:

- **Fuller Start** – Determines the position along the blade's length where the fuller begins, defining the starting point of the groove.
- **Fuller End** – Determines the position along the blade's length where the fuller ends, defining the end of the groove.
- **Fuller Depth** – Controls how deeply the fuller is carved into the blade's thickness, constrained to prevent unrealistic deformation.
- **Fuller Width** – Defines the width of the fuller across the blades surface, influencing its definition and overall form.
- **Circle Radius** – Determines the size of circular fuller holes, controlling the diameter of the opening.

- **Fuller X Position** – Specifies the horizontal placement of the fuller across the blade's width, determining how it sits from the centreline.
- **Fuller Falloff** – Controls how smoothly the fuller blends in and out at its start and end points, shaping the transition along the blade's length.

Texture Parameters:

- **Base Material** – Determines the metallic colour, patterning and reflective properties within the shader, shaping the blades visual appearance.
- **Effect** – Defines the additional surface effects applied over the base material, including elements such as rust, mud and blood.
- **Effect Amount** – Controls the intensity of chosen effect applied across the blade's surface, influencing its visual impact.

Hilt Creation Parameters:

- **Point count** – Determines how many points are used to define the guard's shape.
- **Spline length** – Controls the overall length of the guard.
- **Shape Curve** – Defines the guard's curvature based on an animation curve.
- **Max height** – Controls the maximum height the guard can reach.
- **Sample count** – Determines the level of detailed used when generating the guard.
- **Width** – Controls the width of the guard.
- **Thickness** - Controls the guard thickness.
- **Grip scale** – Determines the overall length of the grip
- **Handle X Position** – Determines the Hilts position relative to the blade, along the X Axis.

3.3. Creating the Silhouette

To create the blade's overall form a is needed silhouette which represents a 2D profile of the blade, this serves as a base for creating the final 3D model. Establishing this silhouette early in process enables rapid prototyping and ensures that the system produces visually appeal and structural coherent sword shapes.

To construct the silhouette the system first generates a spline, this defines the centre line of the blade. Splines were chosen because they provide smooth, continuous curvature that can be precisely controlled through a small number of parameters, a technique widely used in procedural modelling due to its stability (Smelik et al., 2014). The spline provides the underlying axis that determines the overall curvature and flow of the sword. Once this centre line is created, predefine parameters such as blade length, taper, curvature intensity and width are evaluated by applying their values directly to the spline, modifying its shape accordingly. These parameters are then validated against a set of constraints to ensure that the resulting shape remains functional and visually appealing, where visual appeal is defined by smooth curvature, proportional taper, consistent width profile and preventing any unintended irregularities in the blade's silhouette.

Once the parameters pass the appropriate checks, including ensuring that the blade length, taper, curvature intensity and width remain within acceptable bounds, produce smooth transitions and does not introduce irregularities. The silhouette is generated by sampling the width profile at each spline point along the centre line, offsetting each point to the left and right based on the width value, and connecting these points to form the final silhouette, which is then used as the basis for 3D generation.

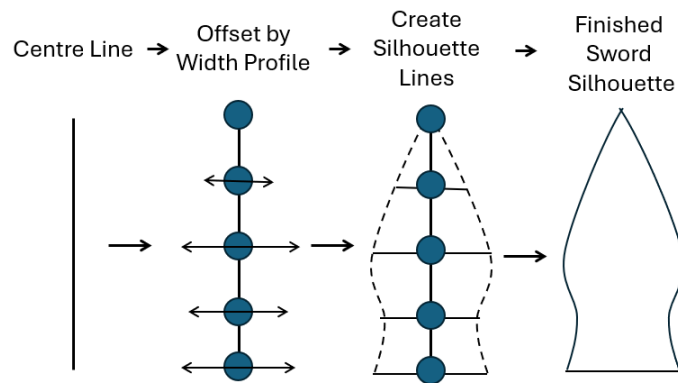


Figure 1: Creating A Sword Silhouette

Blue – Centre Line, **Green** – Silhouette, **Orange** – Offset Direction

3.4. Creating the 3D Blade

Once the base silhouette is established, the next step is to generate a 3D blade model. This is achieved by extruding the 2D profile and assigning it a defined thickness. In practice this is done by getting two parallel planes positioned symmetrically on either side of the centre line, offset by the thickness value along the new axis.

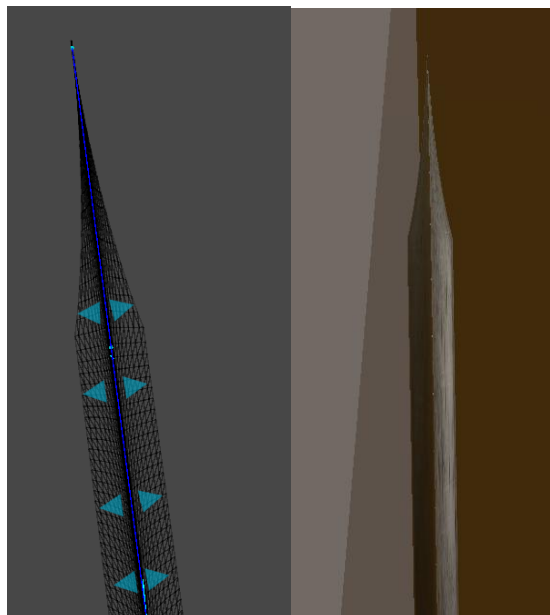
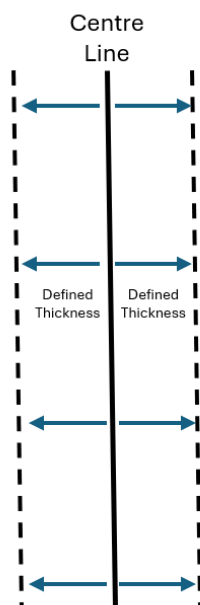


Figure 2: Extruding The 2D Profile

While this process successfully produces a volumetric blade, it introduces an immediate limitation: the resulting blade has flat rectangular edges and a blunt tip. In other words, the extrusion alone does not capture the tapered, sharpened characteristics expected of a functional blade and further refinement is required to achieve a realistic blades form.

To produce the sharp edges characteristic of a forged blade, the mesh's edge vertices are translated inward along the thickness axis so that they converge toward the centre. This creates the classic hard-edge profile.

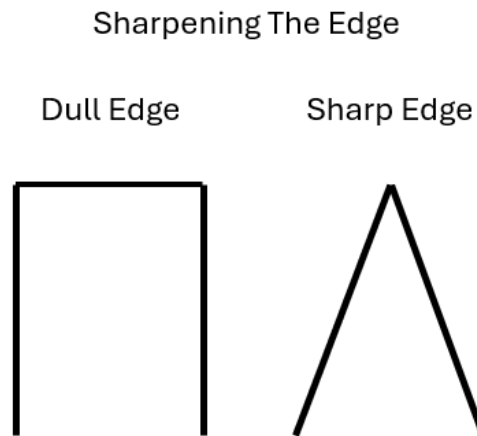


Figure 3: Sharpening The Edge

For a more advanced and controllable result, a bevel can be introduced: instead of meeting at a single point, the vertices form a small intermediate edge before tapering to the centre. This approach allows the sharpness of the blade to be parameterised, enabling the system to generate anything from a razor-sharp edge to a deliberately dulled profile. Bevel parameterisation is essential because bevel angles and thickness directly influence sharpness and geometric profile of a blade, as demonstrated in historical studies (Williams, 2012).

Bevel Variations

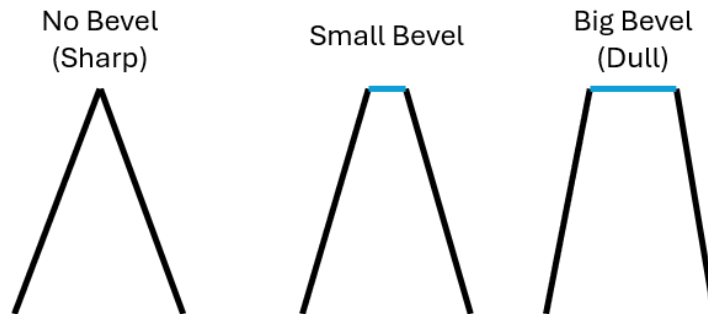


Figure 4: Bevel Variations

To generate a sharp blade tip within the procedural spline-based mesh system, a slightly more complex approach is required compared to forming the blade's edges. The final segment of the spline must be progressively narrowed along both the width and thickness axes, allowing the geometry to converge smoothly toward a single point. During this process, the left and right vertex are brought together and merged to prevent overlap. This convergence must also preserve the curvature defined by the underlying spline, ensuring that the tip maintains structural coherence reflecting Oakeshott's observation that historical blades exhibit continuous taper rather than abrupt geometric transitions (Oakeshott, 1991).

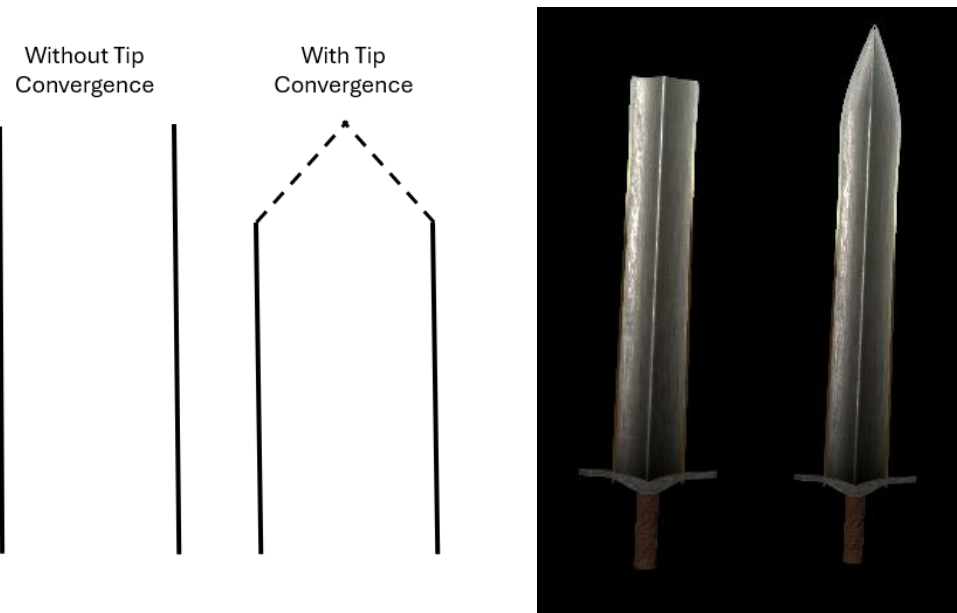


Figure 5: Converging The Tip of The Blade

3.5. Fullers and Holes

To create fullers, the system deforms the blade surface to carve a recessed groove along its length. The fuller's start and end points are taken from the blade's spline, allowing the groove to fade in and out smoothly rather than appearing abruptly. A depth profile is then generated for each cross section of the blade by measuring how far each vertex lies from the fuller's centreline and applying a falloff curve, so the groove is deepest in the middle and tapers toward the edges. The width, depth, and falloff shape are all controlled by user parameters, with the depth scaled to suit the blade's overall size. These parameters were chosen because fuller width, depth and placement are historically significant feature that influence both the structural efficiency and visual character of swords, as documented in historical analyses of blade construction (Oakeshott, 1991; Williams, 2012).

Once this depth map is calculated and smoothed, the deformation is applied by pushing the front face vertices inward and the back face vertices outward along the blade's normal direction. This maintains the blade's thickness while forming a clean, symmetrical channel that follows

the curvature of the blade. The result is a fuller that appears intentional, consistent, and fully integrated into the procedural mesh.

To create holes within the blade mesh, a mask-based approach was used. The system stores a set of predefined positions indicating where each hole should appear, and the corresponding faces are simply omitted during mesh construction. However, removing faces alone results in an undesirable artefact: the interior of the blade becomes visible, producing a hollow, unfinished appearance.

To address this, additional geometry is generated around the perimeter of each hole. Wall faces are constructed by connecting the corresponding vertices on the front and back surfaces of the blade, effectively sealing the interior and giving the hole a clean, intentional boundary. An exception is made when a hole intersects the outer edge of the blade. In these cases, no internal wall is created, ensuring the opening appears natural rather than producing floating or disconnected geometry outside the blade's silhouette.

3.6. Shader

The blade's texture was implemented using Unity's built-in Shader Graph, chosen for its node-based workflow and its ability to visually prototype and iterate on texture behaviours. This approach allowed for rapid experimentation with noise functions, layering strategies, and parameter-driven variation.

Within the Shader Graph, the base texture layer was constructed using a combination of gradient nodes and simple noise. This produced a subtle, nonuniform surface pattern, ensuring that each generated blade exhibited a unique underlying texture. Procedural noise was used because it produces coherent variation without requiring hand painted textures aligning with PCG principles of scalable asset creation (Shaker, Togelius & Nelosn, 2016).

A secondary layer was then introduced to simulate the characteristic detailing found on forged metal. This included smoky diffusion patterns, reflective striations, and minor surface imperfections such as scratches and oil residues. These effects were achieved through a blend of Simple Noise, Gradient Noise, and Voronoi Noise at varying scales, enabling fine control over the distribution and intensity of each feature.

Additional optional effects such as rust, oxidation, dried blood, and mud were incorporated as modular overlays. Each effect was driven by a user-defined intensity and colour parameters, with a simple noise function used to introduce natural variation in coverage. This allowed the shader to produce convincing environmental wear while maintaining procedural flexibility.

All gradients, noise scales, colour values, and effect intensities were exposed as parameters, enabling the user interface to dynamically adjust the shader's appearance based on the blade's generated attributes.

3.7. Rendering

To visualise the procedurally generated blade, a dedicated rendering scene was constructed within Unity. This scene included the user interface and its associated generation script, a camera configured for orthographic capture, a directional light to ensure consistent illumination, a neutral background, and the blade-generation object itself.

The rendering pipeline began with the creation of a 2D silhouette using the spline-based generation script. This silhouette served as the foundation for constructing the full 3D mesh. Once the mesh was produced, a custom shader was applied and parameterised to achieve the desired material response and visual style. The final mesh, with all shader properties configured, was then rendered within the scene.

This setup enabled rapid iteration and reliable visual feedback during development. It also provided a controlled environment for presenting the

generated blades clearly and consistently, supporting both debugging and demonstration purposes.

3.8. Feedback and Survey

User feedback for the project was collected through two separate surveys, each targeting a different aspect of the system's evaluation.

The first survey focused on assessing the visual credibility of the generated blades. Participants were shown ten sword designs: five produced by the procedural generation system and five created through traditional 3D modelling methods. These 5 models were all sourced from the Unity Asset Store ensuring they were all produced using standard artist driven 3D modelling workflows. Although the model came from different creators, obtaining them from the same marketplace, ensured a baseline of technical quality, topology and compatibility with Unity's rendering pipeline. This provided enough consistency for use as a reference set when comparing against procedurally generated blades.

For each blade, users rated how convincingly it resembled a sword on a 1–5 scale. They were also invited to provide qualitative comments regarding their overall impression of the design and to suggest any changes they felt would improve its appearance.

The second survey evaluated the usability of the software itself. Participants were asked to interact directly with the application before responding to a series of questions concerning ease of use, clarity of the interface, potential improvements, and whether they were able to create the type of blade they intended.

Both surveys were distributed online via email, through Microsoft Forms to maximise accessibility and streamline data collection. The software was made available through Itch.io as a browser-based build, enabling participants to test the system without requiring a local installation. A total of 17 participants completed the first survey and 6 completed the second.

These participants were recruited, through university group chats, word of mouth and posts made on LinkedIn.

Two separate surveys were used rather than a single combined instrument due to hardware considerations. The procedural generation software runs only on PC, meaning some users may not have been able to complete a full usability test. By separating the visual evaluation survey which can be completed on any device from the software interaction survey, a broader and more diverse participant pool could be reached while still gathering meaningful usability data from those with compatible hardware.

Chapter 4 Results

Data was gathered from participants via two surveys. The first consisted of a structured visual questionnaire in which 17 participants compared images of procedurally generation and hand modelled blades, rating each on a scale of 1-5 and providing feedback on their overall appearance and realism.

The second consisted of 7 participants answering a structured usability questionnaire completed after participants used the procedural generation system directly and created their own blades. These questions included asking their overall thoughts on the project, if they managed to make the blade they wanted, if there was a good variety of random blades and if they looked visually appealing, if they had any issues and if they had any recommendations for improvements.

Together these two surveys were designed to evaluated both the visual output and the usability of the program.

4.1 Visual Questionnaire Data

Q1	Please rate this sword out of 5 (1-Bad, 2-Not Great,3 - Okay, 4-Good, 5 - Excellent)
Q2	I believe the attached image is a realistic depiction of a sword
Q3	If you disagree with the above statement, please state why
Q4	What are your overall thoughts on the sword?

Table 1: Questionnaire Survey Table

The questions asked as part of the questionnaire were intended to be used to gather user feedback and assist on the breakdown of the project's statistics and success.

The visual questionnaire was completed by a total of 17 participants. Each participant was shown ten sword images, five procedurally generated and five hand modelled before being asked to evaluate each one based on the questions above. The order of the images show was kept the same to ensure feedback could be gathered correctly.



Figure 6: Generated Sword Images

A few participants had personal preferences on swords, with varying opinions on what counted as a sword, with some choosing one type of sword like a gladius or long sword and basing their thoughts on that style and others basing their opinion on a more general idea of swords.

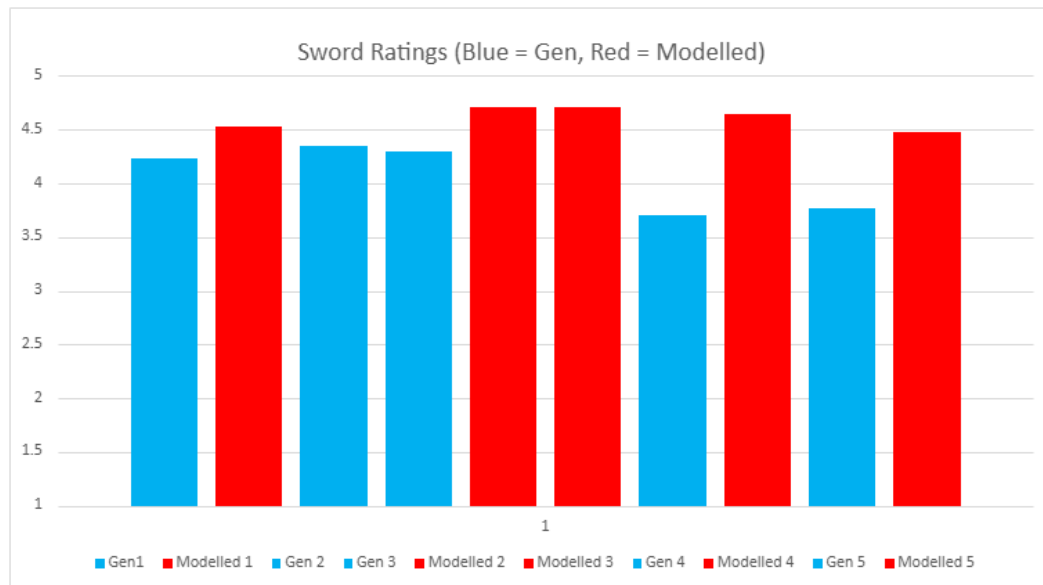


Figure 7: Visual Rating of Sword Results

The results from question 1 (Q1) show that both hand modelled and procedurally generated swords were rated positively with all average scores above 4/5. This indicates participants generally believed all swords were visually credible representations of real weapons.

The average modelled sword rated 4.6125/5.0 and the average generated sword rated 4.175/5.0. However there 2 generated sword's that were riskier and more damaged, having effects such as rust and surface wear, so this may have change how participants viewed the questions, thinking a damaged sword was worse purely because it was damaged. However, if we remove these two blades from the results the outcome is considerably different, especially if we look at the best generated blade compared to lowest rated modelled blade. The generated blade getting 4.4375/5 and the modelled blade getting 4.5/5

which is a much narrower margin. This suggests that in terms of output quality the procedural system is broadly competitive with traditional modelling methods and that the gap is partially a produce of stylistic choices within the generated set rather than the fundamentals of the generation process.

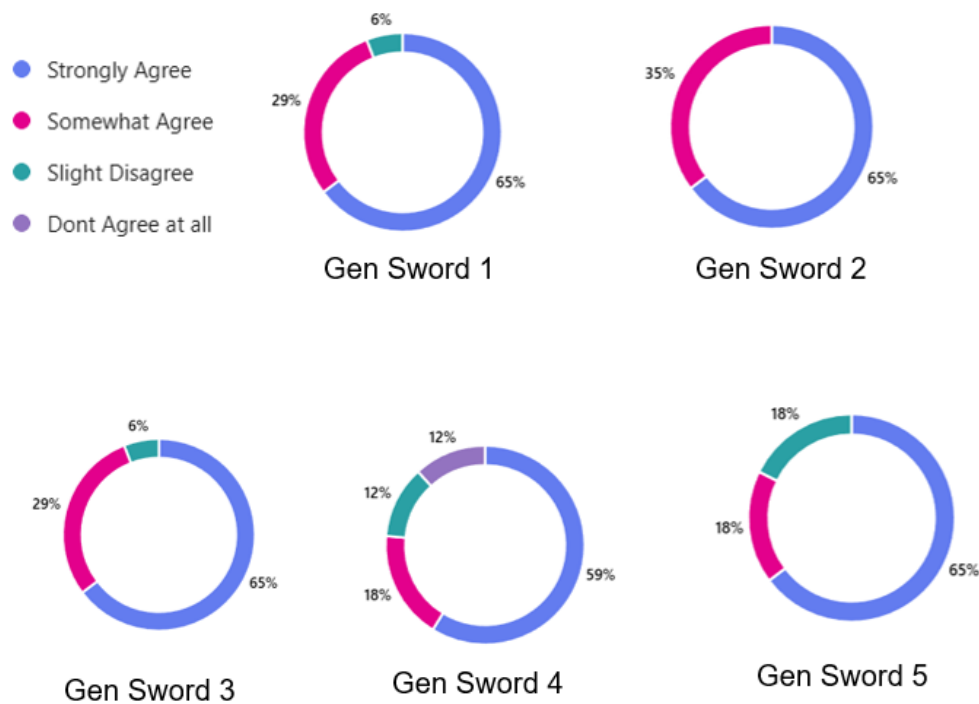


Figure 8: Pie Charts of Generated Swords Believability

Responses to question 2 (Q2), which asked participants whether they believe the image was a realistic depiction of a sword, were mostly positive across all 5 generated blades, though a notable variation is seen between each example.

Generated sword 1 and Generated Sword 4 produced near identical response distributions, with 65% of participants strongly agreeing and 29% somewhat agreeing that each counted as a realistic sword, resulting in an agreement rate of 94% for both blades, with only a single participant

selecting that they slightly disagree in each case. Generated sword 2 was the strongest performer with 65% percent strongly agreeing and 35% somewhat agreeing, and no one selecting that they disagree. This suggests that at least one generated blade was perceived as entirely credible by all participants with no doubts about realism at all.

Generated sword 5 maintained a strong agreement rate of 83% with 65% strongly agreeing and 18% somewhat agreeing, though 18% of participants selected slightly disagree indicating some reservation. Whilst Generated blade 4 was the weakest performer from the group, with an agreement rate of only 77%, 59% being strongly agreed and 18% somewhat agreeing, whilst 12% slightly disagreed and a further 12% didn't agree at all. This is the only sword across the whole group to receive any didn't agree at all results, making it a clear outlier. Consistent with the ratings in question 1(Q1), this blade is one of the two intentionally damaged designs, included in the group, and the disagreement responses like reflect the participants reacting to the worn and degraded appearance of the blade rather than the failure in the underlying generated geometry.

When the three non-damaged swords are considered in isolation the average agreement rate is rises to 96%, which is directly competitive with the agreement rate of 100% for the hand modelled blades. This reinforces that stylistic design choices, rather than geometric deficiencies, account for the bulk of the difference in perceive realism between the two groups.

4.2 Program Questionnaire Data

Q1	After testing the project, what are your overall thoughts?
Q2	Did you manage to make the blade you wanted?

Q3	When clicking randomise did a good variety of blades appear? and what types did you see?
Q4	When clicking randomise did the blades look good?
Q5	Did you have any issues? If so, what were they?
Q6	Finally, do you have any recommendations for improvements?

Table 2: Program Survey Table

3. Did you manage to make the blade you wanted?

7 Responses

ID ↑	Name	Responses
1	anonymous	Yes
2	anonymous	Yes, I wasn't terribly familiar with a lot of the terminology when setting paramaters but it didn't take much fiddling to find out what most of the options changed
3	anonymous	Yes
4	anonymous	yes
5	anonymous	I did yes
6	anonymous	I would say I did thought mainly through trial and error and randomise.
7	anonymous	Yes

Figure 9: Blade Creation Pie Results

All 7 participants reported they were able to create the blade they intended, either through the parameter adjustment method or by using the random generation feature as a starting point and refining the blade from there. This represents a 100% success rate for the core design, suggesting that the parameter set is sufficient to support the rand of designs the users wanted to create. Some participants did note that unfamiliarity with sword terminology, such as fullers and bevels, slowed their UI navigation though this did not stop them from creating their desired blade.



Figure 10: Randomisation Results Pie Chart

Responses regarding the randomisation feature indicated that the system produced a wide variety of visually appealing blades. Participants reported seeing a wide range of outputs, including straight swords, curved swords, single edged designs and various more experimental blades. When asked whether the randomly generated blades looked good, 43% of participants stated that all blades they saw were visually appealing whilst the remaining 57% stated that most, but not all the generated blades were visually appealing. No participant reported that all the randomly generated blades weren't visually appealing. The occasional outlier blade, as expected from the chance for exaggerated proportions was noted by several participants, though this was generally regarded as acceptable, given that the randomisation function is intended to produce the full range of possible blades including unusual designs.

Reported issues were primarily interface related rather than generative. The most common being that participants weren't familiar with sword terminology. One participant did report a minor performance issue when trying to generate their blade, though this was on a web build which hadn't been fully tested compared to the executable. No participants reported any issues that prevented them from generating their desired blade.

Participants did suggest some improvements which mainly focused on three areas. First there were requests for an undo button to allow

participants to fix mistakes easier. Second, improvements to the user interface, though this is purely a visual suggestion. Finally, suggestions on being able to edit the lighting in the scene to see all aspects of the generated swords.

Chapter 5 Discussion

5.1. Survey Session

The Surveys were sent out to participants via email and text messages, which included a link to the Microsoft forms survey, as well as a link to the itch.io page where the project could be run on a web browser or downloading. Participants were sent links to both the Visual and Program questionnaire surveys but had the option to do one or both surveys.

The visual questionnaire survey had a total of 17 participants, giving feedback and asking questions. The Program Questionnaire Survey had a total of 7 participants who tested the program before answering the survey, giving their feedback and asking any question.

5.2. Visual Questionnaire Data Analysis

Looking at data gathered from the Visual Questionnaire, hand modelled blades have a higher rating than generated ones, however this doesn't mean that the program isn't working as intended. As the generated blades don't have the extra finer details like engravings that hand modelled blades have.

The average ratings show only a small difference between generated and hand modelled blades, showing the system produces consistent credible results, The generated blades on average have a rating 4.175 (83.5%) and modelled blades a rating 4.6125 (92.25%), with a difference of 8.75%. This shows that whilst modelled blades look slightly better, the program can generate consistent reliable blades quickly.

Something to keep in mind is that these ratings do not consider that two of the generated models were damaged blades which could have skewed the results, based on the user's interpretation of a good blade not being damaged. Another way to look at the results is to compare the best generated blade to the worst modelled blade.

This lets us view the results differently as the generated rating is now 4.4375/5 (88.75%) compared to the modelled rating of 4.5/5 (90%), having a much smaller difference of 1.25%. This difference shows the potential of procedurally generating the blades, with a few refinements potentially making it so the blade will rate on par or higher than a modelled blade.

These findings align with the broader argument made by Shaker, Togelius and Nelson (2016), who note that procedural content generation does not need to match the absolute ceiling of hand-crafted content to be viable, it just needs to meet an acceptable quality threshold consistency. The results suggest that this system does in fact meet that threshold. The gap that remains is mostly due to the absence of fine surface detailing such as engravings and hilt details, rather than any fundamental weakness in the approach taken for the procedural blade generation. This is an important distinction to keep in mind, as the generation logic itself is good and the quality missing is mainly a surface level issue which further refinement can fix.

In addition, it is worth noting that participants' responses to question 2 (Q2) of the Visual Questionnaire were mostly positive with the three non-damaged blades receiving an average agreement rate of 96% on visual realism, which is very competitive with the 100% agreement rate across the hand modelled set. This further supports the conclusion that the core generation process produces plausible results. The feedback gathered through question 3 and 4 (Q3 and Q4) of the visual questionnaire reinforce this with no participant reporting negative structural issues with the tapering, silhouettes, or mesh.

A further limitation of the visual evaluation is that participants brought their own prior ideals about what makes a viable sword. Some participants evaluated blades primarily based on one style of historic blade such as a gladius or longsword and rated designs that didn't match that style lower even when they were geometrically coherent. This introduces a degree of subjectiveness that is difficult to control. Further evaluations could address this by having users view a defined reference of a historical sword and rating the blades viewed based on its geometric accuracy compared to the historical sword.

When considered alongside the comparison Volcano (Daniele Loiacono, 2015), these results carry additional significance. Volcano generates visually diverse sword silhouettes but does not model the underlying anatomical geometry of the blade, producing no fullers, bevels, cross sections or tip convergence logic. The blades produced in this system, include all these features and were rated above 4/5 across all generated examples. This suggests that the additional geometric detail introduced in this project contributes to the visual quality, and that parametric rules-based approach to full mesh generation offers a clear advancement over silhouette only systems.

5.3. Program Questionnaire Data Analysis

When looking at data gathered from the Program Questionnaire, it shows that program is both functional and easy to use. The 100% success rate of all 7 participants being able to create their ideal blade, indicates this, and demonstrates that the parameter set chosen supports the creation of a wide range of blade designs, regardless of the user's prior knowledge of sword terminology.

The terminology issue is worth looking at more closely, as several participants noted that terms such as fullers, bevels and hilts caused

some confusion. Importantly it made the process of creating their ideal blade take longer, even though didn't stop them from creating it. This distinction matters because it suggests that the generation logic and parameters are intuitive enough that users could navigate them through experimentation even when unfamiliar with wording. This is consistent with the findings of Yannakakis and Togelius (2018), who note that effective procedural tools should allow non expert users to produce meaningful results without requiring a deep amount of knowledge of the subject that is being generated.

The randomisation feature performed well with all participants reporting that at least some generated blades were visually appealing and other 43% noting that all generated blades were good. The other 57% noted that some outlier blades had more exaggerated features and weren't as visually appealing, an intentional behaviour of the randomisation system, which is designed to explore the full range of blades, typical and atypical. This is consistent with the way procedural generation is discussed in the literature: Hendrikx et al. (2013) noting that the value of procedural systems lies in their ability to produce both typical and atypical outputs, with the latter serving as creative inspiration rather than production ready assets.

Improvement requests for the software included adding an undo button, providing options to change the lighting in the scene, and refining the UI. However, these suggestions relate to usability rather than the generation process itself. In contrast, one suggestion for improving the generation process was to increase the level of texture variation. This was overlooked as the texturing system was implemented late into development, however, is easily achievable by implementing random seeds into the existing noise functions, as the shader pipeline is already in place. This represents a low effect but high impact improvement for further development.

5.4. Project Findings

Based on the results of the surveys, several key findings emerge regarding the effectiveness and usability of the procedural sword generation system.

First, the visual evaluation indicates that the system can produce swords that users perceive as credible, and visually accurate. Although hand modelled blades received slightly higher ratings, the margin between the two are relatively small, and the strongest generated example, approached the quality of the weakest hand model example. This suggests that the core generation process is functioning effectively, and that the remaining gap in perceived quality, is most likely due to the absence of finer surface details and use of a simple place holder hilt generation system, rather than fundamental any fundamental issues with the generated blade forms.

Second, the usability feedback demonstrates the software generally considered easy to use, with all participants able to creates their intended blade, despite some confusion around blade terminology. This indicates that the interface and workflow are broadly accessible, though improvements such as an undo function, adjustable lighting and a more polished UI, could enhance the overall user experience.

Third, the random generation feature was positively received, with all users reporting that the system produced at least some visually appealing swords, with nearly half the users stating that all generated swords were good. This highlights the reliability of the underlying procedural rules and supports the idea that the system can create a diverse range of believable swords without manual intervention.

Finally, the feedback also identified areas of future refinement, particularly relating to the texture variation. As the component was

implemented later in the development process, its limitations were expended and the suggestion to implement random seeds into the noise functions provides a clear and achievable path for further improvement.

5.5. Summary

Together these findings show that the project successfully demonstrates the viability of using a procedural generation system to create swords, with strong foundations in silhouette construction, consistent user perceived quality, and an accessible workflow. The remaining limitations are primarily aesthetic rather than structural, indicating that further refinement passes could improve the generated assets to a level comparable with manually modelled alternatives.

5.6. Evaluation

In evaluation I believe the program, successfully creates visually appealing swords, however there is room for improvement, this includes refining the random generation system, renaming variables for those that aren't aware of sword terminology, and improving upon the hilt generation system.

The success also shows in the generated blade ratings, whilst lower than modelled blades, time hasn't been considered. The ability to create various iterations of swords or even a base template for artists to use, within minutes, improves development time significantly. This is especially the case when comparing the hours it usually takes to design a hand modelled blade, let alone multiple when creating a game.

Whilst the users were asked to only rate the blade segment of the sword, however some users did comment on the hilt of the blade. This could have swayed the rating in favour of the hand modelled blades, as hilt generation was seen as a secondary feature in the program and hasn't been fully developed.

In retrospect, taking both the hand modelled blades and generated blades and using the same hilt for all images could have led to a fairer rating system.

Chapter 6 Conclusion and Future Work

6.1. Main Conclusion

This project set out to investigate whether a procedural content generation system can be used to create visually appealing and gematrically accurate swords, that can be suitable for game development. The results of both the visual and program questionnaires demonstrate that system successfully achieves this goal.

User Consistently rated the generated blades as believable and visually accurate, with the small margin between the generated and hand modelled examples, showing that the core generation process is robust and effective. The findings also show that the system can produce a wide range of appealing swords, with the random generation feature performing reliably across users.

Beyond visual quality, the project also highlights the practical advantages of procedural content generation for asset creation. The ability to produce multiple sword variations within a short amount of time represents a significant improvement in development efficiency, when compared to the time heavy process of hand modelling. This reinforces the broader value of procedural methods, as a scalable and flexible alternative for generation large asset libraries in game development.

However, the project also identified some areas that require further refinement. The absence of finer surface details, the simplicity of the generated hit system, and the limited texture variation all contribute to the remaining gap in visual quality. These limitations are aesthetic rather than

structural, suggesting that the underlying generation framework is robust and that future improvements can be achieved through targeted refinement rather than changes to the fundamental system itself.

Overall, the project demonstrates that procedural generation is a viable and effective approach for creating sword assets, offering both visual credibility and gains in production efficiency. With continued development especially in texturing, detail passes and hilt design, the system has the potential to produce assets on the same level or surpass manually modelled alternatives, further supporting the role of procedural techniques within modern game development.

6.2. Implications

The outcome of this project has several implications for procedural content generation and practical asset development workflows. First, the results demonstrate that procedural methods can reliably produce visually credible sword assets with little manual intervention. This suggests that similar silhouette driven generation techniques could be applied to other categories of weapons such as axes, spears, and shields offering a scalable alternative to traditional modelling pipelines.

Second, the efficiency gains within the project, highlight the potential for procedural systems to reduce production time within game development. The ability to generate multiple variations within a short period of time enables rapid iteration, supports prototyping and provides artists with foundations to build upon. This aligns with the industry trends towards hybrid workflows, where procedural tools generate base forms that are later refined through manual passes.

Third, the findings indicate that user perception of procedural assets is strongly influenced by surface detail and presentation, even when the underlying form is visually credible. This suggests that procedural systems do not need to replicate the full complexity of hand-crafted models to

achieve convincing results. Instead, targeted refinement in areas such as texturing, detailing and secondary components such as the hilt, is sufficient to elevate the outputs in production quality.

Finally, the project highlights the importance of usability in procedural tools. Participants were able to create their intended blades despite limited knowledge of sword terminology. This suggests that procedural tools systems can be accessible to those without experience using it. However, the feedback also shows that interface clarity and terminology choices significantly affect user experience, implying that future procedural tools should prioritize intuitive design alongside technical capabilities.

6.3. Future Work

Future development of the program could focus on expanding the system beyond blade generation by implementing an in-depth fully procedural hilt generation feature, instead of the placeholder that current exists. This would allow complete swords to be produced, significantly increasing the variety and creative possibilities available to users.

In addition, the randomisation feature would benefit from being refined. Improving the consistency of generated blades and introducing user configurable presets, so only specific weapon types are randomly generated, such as a katana, gladius, short sword or other historically grounded styles, this would make the tool more adaptable for projects set within a particular genre or time period. This would allow developers to generate controlled variations within a defined style.

Further improvements could be made to the shader and texturing pipeline. Introducing the possibility of random seeds for the noise-based textures, with the options for additional features such as engravings, and additional customisable textures being considered, as these would help close the visual gap between generated and hand modelled assets.

These improvements would also support greater aesthetic variation, without requiring major changes to the current generation logic.

The user interface could be updated to improve visual clarity and usability. While UI changes do not directly affect the quality of the generated swords, they would contribute to a more polished and accessible user experience.

Finally, once a procedural hilt generated has been implemented, the program could be expanded for additional weapon types, such as spears, axes or polearms. This would allow for the project to evolve into a broader procedural weapon generation system capable of producing a wide range of assets for use in game development. Further studies would be required to evaluate the effectiveness of the system, especially when applied to more complex weapon categories.

References

- Ahearn, L., 2013. *3D Game Environments*. s.l.:Focal Press.
- Bartels, R. B. J. & B. B., 1983. *An Introduction to Splines for Use in Computer Graphics*. [Online]
Available at: <https://cs.uwaterloo.ca/research/tr/1983/CS-83-09.pdf>
- Blair, C., 1958. *European Armour: circa 1066 to circa 1700*. London: B. T. Batsford Ltd.
- Bungie, 2017. *Destiny 2*. Bellevue: Bungie.
- Bungie, 3. I., 2001. *Halo*. Redmond: Microsoft Game Studios.
- Carpenter, N., 2025. *Borderlands defined the looter-shooter genre. Borderlands 4 builds*. [Online]
Available at: <https://store.epicgames.com/en-US/news/borderlands-randy-pitchford-interview-borderlands-4>
- Clements, J., 1998. *Medieval Swordsmanship: Illustrated Methods and Techniques*. Boulder, CO: Paladin Press.
- Daniele Loiacono, R. M. , M. P., 2015. *Volcano: An Interactive Sword Generator*. [Online]
Available at: <https://re.public.polimi.it/retrieve/e0c31c09-634e-4599-e053-1705fe0aef77/07377226.pdf>
- David Braben, I. B., 1984. *Elite*. Cambridge: Acornsoft.
- Ebert, D. M. F. P. D. P. K. & W. S., 2003. *Texturing & Modeling: A Procedural Approach*. 4th ed. San Francisco: Morgan Kaufmann.
- Extremes, D., 2013. *Warframe*. London, Ontario: Digital Extremes.
- Games, H., 2016. *No Man's Sky*. s.l.:Hello Games.
- Georgios Yannakakis, J. T., 2011. Experience-Driven Procedural Content Generation. *IEEE Transactions on Affective Computing*, 2(3), pp. 147-161.
- Hendrikx, M. M. S. V. D. V. J. & I. A., 2013. *A Survey of Procedural Content Generation Techniques for Games*. [Online]
Available at: <https://atlarge-research.com/pdfs/2013-hendrikx-procedural.pdf>
- McCrae, S., 2025. *Borderlands 4 laughs in the face of Borderlands 3's 1 billion guns as Gearbox says there's 30 billion this time, which the devs*

kept track of using a Matrix-style "wall" of weapons. [Online]

Available at:

<https://www.gamesradar.com/games/borderlands/borderlands-4-laughs-in-the-face-of-borderlands-3s-1-billion-guns-as-gearbox-says-theres-30-billion-this-time-which-the-devs-kept-track-of-using-a-matrix-style-wall-of-weapons/>

McEvoy, S., 2024. *CD Projekt spent over \$120m to save Cyberpunk 2077.* [Online]

Available at: <https://www.gamesindustry.biz/cd-projekt-spent-over-120m-to-save-cyberpunk-2077>

Michael Toy, G. W. K. A., 1980. *Rogue.* s.l.:A.I. Design, Epyx.

miHoYo, 2020. *Genshin Impact.* Shanghai: miHoYo.

Noor Shaker, J. T. M. J. N., 2016. *Procedural Content Generation in Games.* s.l.:Springer.

Oakeshott, E., 1964. *The Sword In The Age Of Chivalry.* [Online]

Available at:

https://books.google.co.uk/books?hl=en&lr=&id=TYcBwoeWwp0C&oi=fnd&pg=PA25&dq=+sword+designs&ots=T9hca_rIFN&sig=Kn7zvaEphgkYb8eJxN_5aNpbMj0&redir_esc=y#v=onepage&q&f=false

Oakeshott, E., 1991. *Records of the Medieval Sword.* Woodbridge: Boydell Press.

Perlin, K., 1985. An Image Synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3), p. 287–296.

Perlin, K., 2002. Improving Noise. *ACM Transactions on Graphics*, 21(3).

Red, C. P., 2020. *Cyberpunk 2077.* Warsaw: CD Projekt.

Ruben M. Smelik, T. T. R. B. B. B., 2014. *A Survey on Procedural Modelling for Virtual Worlds.* [Online]

Available at:

https://www.researchgate.net/publication/320722498_A_Survey_on_the_Procedural_Generation_of_Virtual_Worlds

Software, G., 2009. *Borderlands.* s.l.:2K Games.

Software, G., 2025. *Borderlands 4.* s.l.:2K Games.

Studios, M., 2011. *Minecraft.* s.l.:Mojang Studios.

Studios, T. B., 2021. *Chivalry 2.* Toronto: Tripwire Presents.

Studios, W., 2018. *Kingdom Come: Deliverance*. Prague: Deep Silver.

Ubisoft, 2017. *For Honor*. Montreal: Ubisoft.

Williams, A., 2012. *The Sword and the Crucible*. Leiden: Brill.

Wood, A., 2024. *It's taken 4 years and roughly \$900 million, but Genshin Impact is a better open-world RPG than ever after update 5.0*. [Online]
Available at: <https://www.gamesradar.com/games/open-world/its-taken-4-years-and-roughly-dollar900-million-but-genshin-impact-is-a-better-open-world-rpg-than-ever-after-update-50/>

Worley, S., 1996. *A Cellular Texture Basis Function*. [Online]
Available at: <https://dl.acm.org/doi/epdf/10.1145/237170.237267>

Yannakakis, G. N. T. J., 2018. *Artificial Intelligence and Games*. s.l.:Springer.